41052 Programming Assignment 2: Selection Algorithms

Michael Lunn - 13917657
Benjamin Hood - 25478064

Table of Contents

Table of Contents **1**
1. The problem and the algorithms **2**
1.1 Problem statement 2
1.2 Quickselect (baseline) and std::nth_element 2
1.3 Median of medians (BFPRT), primary author Michael Lunn 2
1.4 Floyd-Rivest, primary author Ben Hood 3
1.5 Introselect 3
2. Theoretical comparison and predictions **4**
2.1 On paper 4
2.2 Predictions 4
3. Experimental methodology **6**
3.1 What we measured 6
3.2 Input families 6
3.3 Testing environment 7
3.4 Fairness decisions 7
3.5 Reproducing 8
4. Results and analysis **9**
4.1 Does the algorithm with the fewest comparisons win? 9
4.1.1 Fewest comparisons is best 9
4.1.2 Greater variance in quickselect variants 9
4.2 Which inputs break things? 10
4.2.1 BFPRT immune to bad input 11
4.2.2 Organ Pipe 11
4.3 How does the target rank matter? 12
4.4 Effect of the McIlroy adversarial input 14
5. What surprised us **16**
5.1 Fewer comparisons doesn't always mean less time (Michael) 16
5.2 Floyd-Rivest does more work on reverse-sorted input than on random input (Ben) 17
6. What we'd do next **18**
7. Individual contributions **19**
7.1 Michael Lunn 19
7.2 Ben Hood 19
8. AI use **20**
8.1 AI Use Disclosure 20
8.1.1 Michael Lunn 20
8.1.2 Ben Hood 20
8.2 Where AI was wrong, unhelpful or misleading 20
8.3 AI in group work 21
References **22**

1. The problem and the algorithms

1.1 Problem statement

Given an array of n elements and an index k with $0 \leq k < n$, the selection problem asks for the k-th smallest element. This is the one that would sit at position k if the array were sorted, counting from 0.

The naive approach would be to sort the array and then take the k-th element in the array, however, sorting the array takes O(n log n), and the problem does not require a full ordering of the array.

This problem can be solved in worst-case linear time, yet in practice, libraries and programmers usually choose algorithms with weaker guarantees, such as quickselect and introselect, whose worst cases are quadratic or O(n log n) despite the average case running in linear time.

This project asks why: what does a guaranteed linear bound actually cost on real inputs, and is there any situation where it is worth using?

1.2 Quickselect (baseline) and std::nth_element

Quickselect (Hoare's FIND; Hoare, 1961) reuses quicksort's partition step where a pivot is picked and then the array is rearranged so smaller elements come before it and larger ones after. If the pivot lands at position k, then the k-th smallest element is found. Otherwise k lies on exactly one side, and we recurse on that side only.

The time complexity of quickselect relies heavily on how good the chosen pivot is. In the implementation we are testing, pivots are chosen randomly meaning the algorithm is expected to run in O(n) time on average. However, in quickselect, there is no worst case guarantee, if consistently bad pivots are chosen, each round removes only one element and the cost becomes $O(n^2)$.

All of our own implementations use the same Hoare-style partition but vary in their method of choosing the pivot.

std::nth_element is the C++ standard library's version of the same idea, and we use it as our reference. However, there are different implementations of std::nth_element. We use the libc++ implementation which is also a quickselect however there is one major difference. The pivot is chosen as a median of the first, middle, and last element. We therefore treat it as a separate algorithm, not a copy of ours.

1.3 Median of medians (BFPRT), primary author Michael Lunn

Blum, Floyd, Pratt, Rivest and Tarjan (1973) removed the dependence on luck by building a pivot that is guaranteed to be good. First, the array is split into groups of five and then each group's median is calculated, which is cheap because groups are small. Then the median of those medians is found and used as the pivot.

About half of the group medians are at most the pivot, and each of those has two smaller elements in its own group. At least roughly 30% of the elements are no larger than the pivot so every round discards at least 30% of the array.

The cost satisfies $T(n) \leq T(n/5) + T(7n/10) + cn$. Because $1/5 + 7/10 < 1$, this resolves to $O(n)$ in the worst case, on every input.

The price of a guaranteed strong pivot is extra work in every round, sorting groups and making a recursive call before any partitioning, so its constant factor is large. A central question for this project is how much that guarantee costs in practice.

1.4 Floyd-Rivest, primary author Ben Hood

Floyd and Rivest (1975) aim to choose a pivot that is very close to the answer. To find the k-th element of a large range, the algorithm first recursively selects within a small window of roughly n^(2/3) elements around position k. This helps estimate the true k-th smallest element, allowing the algorithm to partition the full range with about one comparison per element. The target is then closer to one end of the remaining range, so a second round with an even tighter window can finish the selection.

The expected number of comparisons is about $n + \min(k, n-k)$, which is around 1.5n for the median and close to n for the minimum or maximum. This makes Floyd-Rivest a strong candidate for the fewest comparisons among the algorithms we test.

Unlike median of medians, which guarantees linear time in worst case, Floyd-Rivest relies on choosing effective pivots, so structured, patterned and adversarial inputs are useful for testing its performance.

1.5 Introselect

Introselect (from Musser's "introspective" algorithms, 1997) combines quickselect's speed with a safety net. Initially, it runs quickselect but will count how many rounds have occurred. If the count passes a limit of a small multiple of $\log_2(n)$, the pivot rule is clearly struggling, and it switches to a safer algorithm for the remaining range. Our version uses random pivot quickselect, a limit of $2 \cdot \log_2 n$ rounds, and median of medians as the fallback.

On ordinary inputs it behaves exactly like quickselect. In the worst case, an adversarial input can make every round before the limit remove almost nothing, at a cost of about n per round. The linear-time fallback then finishes, so the worst case in our implementation is $O(n \log n)$. We will record how often the fallback fires and on which inputs.

2. Theoretical comparison and predictions

2.1 On paper

Algorithm	Worst case	Expected time	Comparisons for median	Extra memory	Sensitive to
Quickselect (random)	$O(n^2)$	O(n)	~3.39n	O(1)	Only bad luck, or a known seed
BFPRT, g = 5	O(n)	O(n)	Bounded by a large constant × n	O(log n) stack	Little
Floyd-Rivest	$O(n^2)$	O(n)	~1.5n	O(log n) stack	Ordered or structured input, because the window is contiguous
Introselect (BFPRT fallback)	O(n log n)	O(n)	~3.39n, same as quickselect	O(log n)	Adversaries that use up the depth limit
std::nth_element	O(n^2)	O(n)	No clean figure	O(1)	Median-of-3 killers

Table 1: Theoretical Complexity and Performance Characteristics of Selection Algorithms

2.2 Predictions

Random input

- **P1.** Quickselect will make the fewest comparisons and BFPRT the most, because of BFPRT's constant overhead. We are tentative about this (we assumed the same ranking holds for time).
- **P2.** Introselect will be fastest, since it should behave like the better of quickselect and BFPRT.
- **P3.** BFPRT will be slower than quickselect; we did not commit to how much.
- **P4.** std::nth_element will beat our implementations and will be closest to our introselect.
- **P5.** If Floyd-Rivest is implemented properly, it will show fewer comparisons than the deterministic algorithms.

Target rank

- **P6.** On random input with n = 10^7, Floyd-Rivest should use approximately 1.5 comparisons per element when selecting the median, and close to 1 when selecting the minimum. This follows from its expected count of n + min(k, n−k). The median needs more comparisons because the target is far from either end of the array.
- **P7.** Floyd-Rivest may win at ranks near either end. Finding the minimum will cost about the same as finding the maximum.

Input shape

- **P8.** Sorted, reverse-sorted and nearly sorted input will hurt quickselect most and favour BFPRT, whose pivot is guaranteed to be good.
- **P9.** Floyd-Rivest will perform better on sorted and reverse-sorted input than on random input, because the window is likely to contain elements close to their final positions, which makes an effective pivot easier to choose.
- **P10.** Floyd-Rivest will perform worse on organ pipe input, because the values do not follow the same ordering as their positions, so the window is a less reliable estimate of the target.
- **P11.** Sawtooth input will also hurt Floyd-Rivest, because its repeating pattern may disturb the sampling.
- **P12.** All-equal input will hurt no algorithm. All will be about equally fast, since one pass resolves it.
- **P13.** Duplicates in general (including two distinct values) will not hurt any algorithm. On two distinct values BFPRT will do relatively worse, because its pivot work is wasted.

Input Size

- **P14.** BFPRT's overhead will grow with n. We do not expect the fastest algorithm to change between small and huge arrays.

Robustness and the adversary

- **P15.** BFPRT will be the most robust algorithm, because of its linear guarantee. The adversary will not break it, since computing a median of medians overcomes crafted input.
- **P16.** The adversary needs a deterministic algorithm, so we do not expect it to break random-pivot quickselect.
- **P17.** A killer array built for one algorithm will not hurt the others.
- **P18.** Introselect's fallback will fire rarely on normal inputs and frequently on adversarial ones.

3. Experimental methodology

3.1 What we measured

For every combination of algorithm, input instance, target rank and repetition, the benchmark harness records one CSV row with three quantities:

1. Running time: wall-clock nanoseconds for the select call alone, measured with std::chrono::steady_clock. The copy of the input into a work buffer happens before the timer starts.
2. Comparison count: the number of comparator calls, measured in a separate run with a counting wrapper, so the counter cannot distort timings. The counted run uses the same input copy and the same random seed as the timed run, so it makes exactly the same comparisons.
3. Fallbacks: how many times introselect switched to its fallback algorithm (zero for all other algorithms).

Comparison counts include every comparison, including those inside insertion sort and BFPRT's group sorts. Element moves and swaps are not counted.

3.2 Input families

Inputs are arrays of 64-bit integers unless stated. Every generator is a deterministic function of its size, seed and parameters, so any instance can be regenerated exactly. Families with randomness (random, nearly sorted, few-distinct) use 3 independent seeded instances per size. The other families have a single instance.

The main sweep uses $n = 10^3$ to 10^7 in steps of about 3×. The adversary experiment uses $n = 10^3$ to 10^5, since generating a killer array means running the attacked algorithm at its worst case. Strings use $n = 10^4$ and 10^5 with prefixes of 0, 16, 64, 256 and 1024 characters.

Inputs are generated from 10 different families:

1. Random
2. Sorted
3. Reverse-sorted
4. Nearly sorted (sorted and then some swapped at random)
5. All-equal
6. Two distinct values (e.g. 1s and 0s)
7. Many distinct values (e.g. 100 values with many duplicates)
8. Sawtooth: The values 0 to 99 repeat throughout the array (0, 1, 2, …, 99, 0, 1, 2, …). This creates many short ascending sequences. It tests whether algorithms can choose good pivots when the same pattern appears throughout the array.
9. Organ pipe: The values increase to a peak in the middle and then decrease again (0, 1, 2, …, peak, …, 2, 1, 0). Most values appear twice, with the largest values in the middle and smallest at the ends. This tests algorithms that rely on array positions when choosing pivots, since the values are not evenly distributed across the array.
10. Adversarial: McIlroy's adversary (McIlroy, 1999) builds an input that makes one specific deterministic algorithm do as much work as possible for a given n and k. We build killer arrays against quickselect, introselect, BFPRT, Floyd-Rivest and

std::nth_element. Since our implementations of quickselect and introselect use a random pivot, they can only be attacked with a fixed seed, so we attacked them with seed 1.

In addition to these families we also test:

- Strings: Any integer input above converted to strings made of p copies of the letter ‘a’ followed by the integer in fixed-width base 26. This preserves the ordering, so every family carries over, but each comparison must scan the whole shared prefix. This is intended to make comparisons costly to see what effect they have on the results.
- Target rank: The position k (counting from 0) of the element to find, tested at min (k = 0), the 25th percentile, the median, the 75th percentile and max (k = n−1).

3.3 Testing environment

Device	M2 Macbook Air
OS	MacOS 15.4.1
CPU	Apple M2
Core count	8 (4 performance cores, 4 efficiency cores)
Compiler	Apple clang version 17.0.0 (clang-1700.0.13.5)
Compiler flags	-std=c++17 -O2 -Wall -Wextra
Standard library	libc++ 19.1.2, the source of std::nth_element in our results
Repetitions	1 untimed warm-up and 11 timed repetitions per cell; random families use 3 input seeds (2 for strings)

Table 2: Experimental Environment and Benchmark Configuration

3.4 Fairness decisions

The comparison is meant to measure each algorithm’s strategy for choosing pivots, not differences in how carefully each was implemented. Four design decisions support this.

Shared building blocks. All four custom algorithms (quickselect, BFPRT, Floyd-Rivest and introselect) call the same Hoare-style partition and the same narrowing step in algorithms/partition.hpp, and they all finish ranges of 16 elements or fewer with the same insertion sort. Hoare partitioning splits elements equal to the pivot across both sides, so all-equal and few-distinct inputs cannot push any algorithm into quadratic behaviour through the partition alone. Each algorithm uses its published default parameters: g = 5 for BFPRT, a sampling threshold of 600 for Floyd-Rivest, and a depth limit of $2 \cdot \log_2 n$ for introselect. std::nth_element is the deliberate exception. It is libc++’s own code, with its own partition

and cutoff and no depth-limit fallback, so we treat it as a separate reference point rather than as an equal implementation.

Identical inputs. Each input instance is generated once per family, size, seed, prefix and rank, then copied fresh into a work buffer before every run. Each CSV row records a hash of the input, and rows for different algorithms on the same instance share that hash, which shows they received identical data.

Measuring the algorithm only. The timer wraps the selection call alone. Copying the input and computing the expected answer both happen outside it, and compiler fences stop the timed call from being reordered across the clock reads. Comparisons are counted in a separate run with a counting comparator, so the counter's overhead never enters the timings. That counted run reuses the same input and random seed as the matching timed run, so randomised algorithms make exactly the same choices in both.

Correctness before performance. make test checks every algorithm configuration under AddressSanitizer and UndefinedBehaviorSanitizer before any benchmark runs. During benchmarking, the first repetition of every cell is checked again: a[k] must equal the true k-th element and the array must be partitioned around it, and any failure stops the run.

3.5 Reproducing

All commands run from the repository root, on a machine with a C++17 compiler, make, and Python 3 with numpy, pandas and matplotlib.

1. Run make to build the benchmark harness (build/bench) and the generator tests.
2. Run make test to check every algorithm configuration under AddressSanitizer and UndefinedBehaviorSanitizer. Benchmarking should only go ahead if this passes.
3. Run benchmarking/run_all.sh main for the main sweep: every algorithm on every input family, size and rank.
4. Run benchmarking/run_all.sh adv for the McIlroy adversary experiment. It generates one killer array per target algorithm and replays each array against every algorithm.
5. Run benchmarking/run_all.sh strings for the shared-prefix string experiment.

Each phase writes one CSV to results/ (main.csv, adversary.csv, strings.csv) together with a .meta.txt file recording the CPU, compiler, flags and exact command line. Plots and summary tables go to results/plots_<phase>/. Unless overridden, every cell runs one untimed warm-up and then 11 timed repetitions, and random families use 3 input seeds. A cell whose warm-up takes longer than 2 to 3 seconds gets a single repetition, so quadratic cases cannot stall a sweep. Adversary arrays are cached in results/adversaries/ and replayed unchanged, so a rerun tests exactly the same inputs. All randomness comes from explicit seeds, so comparison counts are identical across machines; only timings vary with hardware.

4. Results and analysis

Note: The std::nth_element results in this study use the libc++ implementation provided by Apple Clang on macOS, not the libstdc++ implementation commonly used with GCC on Linux. Upon further research, the relevant difference here is that libc++ implementation does not have a fallback when a certain depth is reached as it does in libstdc++ (Heapsort), meaning in practice, it performs closer to our implementation of quickselect than to introselect.

4.1 Does the algorithm with the fewest comparisons win?

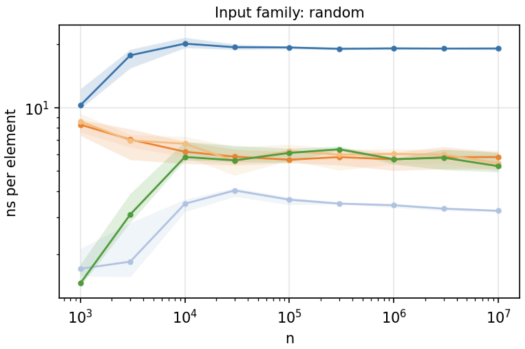


Figure 1: Time per element vs n (random integers, median rank)

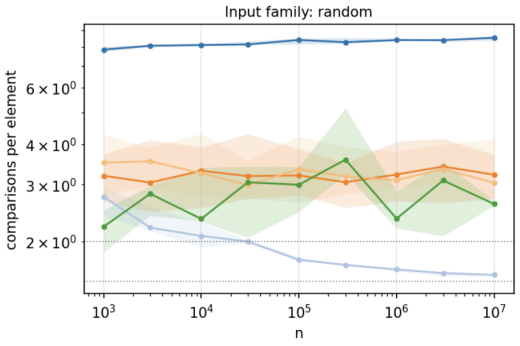
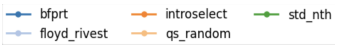


Figure 2: Comparisons per element vs n (random integers, median rank, reference lines at 1.5 and 2)



4.1.1 Fewest comparisons is best

At n = 10^7, BFPRT takes 19.1 ns per element. That is approximately 3.5× quickselect's 5.433 ns, almost 6× Floyd-Rivest's 3.224 ns, and 3.6× std::nth_element's 5.26 ns.

From fastest to slowest the order is Floyd-Rivest, std::nth, quickselect, introselect, BFPRT. This ordering holds when looking at Figure 2 which suggests that the biggest factor in running time is the number of comparisons.

Floyd-Rivest performed best in number of comparisons. In our testing, we found the value had approached the accepted textbook value of 1.5n as n grew.

4.1.2 Greater variance in quickselect variants

The shaded areas in our plots represent the IQR with the upper and lower edges representing the 75th and 25th percentile respectively. BFPRT's comparisons per element stayed constant across individual runs, while the quickselect variants varied much more, as seen by the greater variance in Figure 2. This illustrates one of the key differences between how the pivot is chosen in each of these algorithm families. BFPRT builds its pivot from the

medians of groups of five, which guarantees that at least about 30% of the range lies on each side of it. Every round therefore discards a fixed fraction of the range, whatever the data looks like. Quickselect's pivot quality, by contrast, depends on luck or on the data. A random pivot (as implemented in our quickselect and introselect algorithms) gives a count that can vary significantly from run to run. We expected BFPRT's flat curve to sit well above the quickselect variants' typical counts, because it pays for this stability with extra comparisons in every round.

4.2 Which inputs break things?

Family	BFPRT	Floyd-Rivest	Introselect	Quickselect	std::nth
Sorted	0.824	0.637	1.049	1.170	0.383
Reverse-sorted	0.930	1.627	1.176	1.306	0.671
Nearly sorted	0.840	0.958	1.101	1.170	0.383
All equal	0.694	1.28	0.622	0.658	0.767
Two distinct	1.176	1.719	0.847	0.896	0.767
Few distinct	0.948	1.019	1.089	1.100	0.959
Sawtooth	0.901	1.021	0.982	0.950	2.914
Organ pipe	0.918	9.265	0.882	1.267	15.57

Table 3: Comparisons per element as a multiple of random input performance (n = 10^7, median rank)

Family	BFPRT	Floyd-Rivest	Introselect	Quickselect	std::nth
Sorted	0.434	0.093	0.172	0.195	0.056
Reverse-sorted	0.427	0.282	0.234	0.245	0.110
Nearly sorted	0.472	0.164	0.241	0.260	0.083
All equal	0.310	0.278	0.153	0.165	0.114
Two distinct	0.805	1.410	0.754	0.817	0.621
Few distinct	0.936	1.038	1.046	1.089	1.110
Sawtooth	0.642	0.233	0.332	0.347	0.671
Organ pipe	0.423	1.409	0.171	0.252	2.321

Table 4: Time per element as a multiple of random input performance (n = 10^7, median rank)

4.2.1 BFPRT immune to bad input

As expected, no structured input family meaningfully worsened BFPRT's performance (Table 3 and 4). Its time per element performed no worse than on random input for every family. The quickselect variants, by contrast, varied more across the same families.

This follows from how BFPRT picks its pivot. The pivot is the median of group medians, so its rank depends on every group and not on a few fixed positions. No arrangement of the data can stop it from leaving at least about 30% of the range on each side, so every round discards a fixed fraction of the range. The shared partition also splits elements equal to the pivot evenly, so duplicates cost no extra rounds.

We had predicted that few distinct values would hurt BFPRT most, because its pivot work is wasted when many elements are equal. We did not see this in the data. On reflection, the overhead is a constant cost paid on every input, including random ones, so it should show up as BFPRT being slower than the other algorithms everywhere, not as a worsening on this particular family. The prediction therefore compared the wrong baseline. BFPRT is robust but not cheap so its flat cost sits above that of the quickselect variants on every family, not just ones where that extra cost is not valuable.

4.2.2 Organ Pipe

Organ pipe was the one input that clearly broke algorithms that pick pivots from sampled positions. Comparisons per element rose to about 9.3× the random-input value for Floyd-Rivest and about 15.6× for std::nth_element.

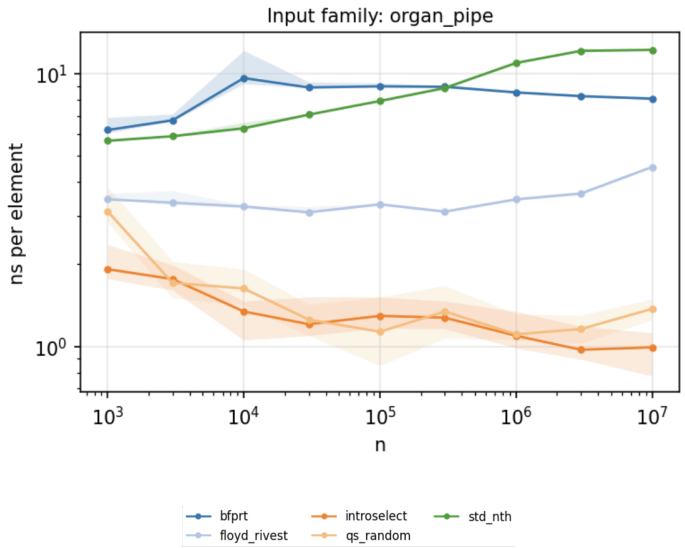


Figure 3: Time per element vs n (organ pipe, median rank)

For Floyd-Rivest this was predicted. Its window is taken from the array around position k, and on organ pipe the values near position k are not near the k-th smallest value, because the values rise to a peak in the middle and fall again. For a median target the window sits at

the peak, so the pivot it produces is far above the target. Partitioning around it discards little, the target stays in a range almost as large as before, and the window makes the same mistake in the next round. The cost accumulates as repeated near-full passes instead of the expected n + min(k, n−k).

We made no prediction for std::nth_element, however, the limitations of the median of three pivot selection becomes evident in these results. Our toolchain's (libc++ 19.1.2) std::nth_element is, as far as we can tell, a median-of-3 (first, middle, and last elements) quickselect with no depth limit or fallback, so it has nothing to stop a bad run. On organ pipe, the first and last are the smallest values and the middle is the peak, so the median is a very small value. Partitioning around it removes almost nothing. This also explains why the cost per element rose with n, even surpassing BFPRT at n greater than 10^6.

This result illustrates the value of a fallback in these algorithms as the organ pipe input clearly breaks a median-of-three quickselect.

4.3 How does the target rank matter?

The rank-sweep results suggest that the target rank influences the number of comparisons for most algorithms, but has little effect on BFPRT. This is consistent with their designs. Quickselect, introselect, Floyd–Rivest, and std::nth_element repeatedly partition the active range and stop once the target rank falls into the resolved partition; therefore, selecting an extreme rank such as the minimum or maximum can produce a different sequence of subproblem sizes and partition work than selecting the median.

Floyd–Rivest is additionally designed to use rank-dependent sampling and narrowing, which can make this effect more pronounced. Introselect follows the same partition-driven process as quickselect until its depth limit is reached, so its rank sensitivity is inherited from that structure.

In contrast, BFPRT uses the median-of-medians procedure to select a guaranteed-quality pivot independently of the target rank, then applies the same deterministic grouping and partitioning framework at each level. Although the final recursion path can still vary slightly with rank, its guaranteed pivot balance makes the total comparison count much more uniform across target ranks.

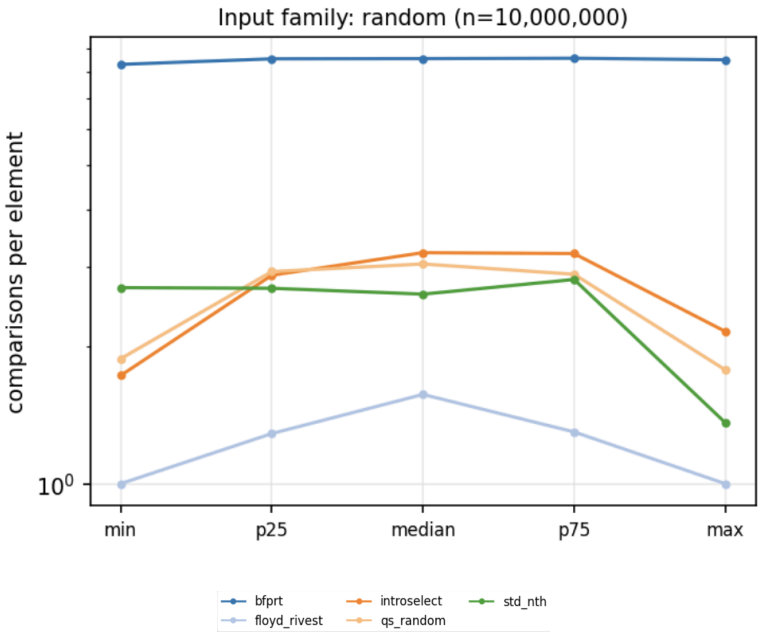


Figure 4: Comparisons per element vs k/n (n = 10^7)

4.4 Effect of the McIlroy adversarial input

Algorithm	Time	Comparisons
BFPRT	0.55	0.91
Floyd-Rivest	16.47	105.81
Introselect	3.705	12.72
Quickselect	894.25	5611.13
std::nth_element	908.15	6265.41

Table 5: Time & comparisons per element as a multiple of the respective random input (n = 10^5, median rank)

The McIlroy adversarial inputs produced substantially different effects across the selection algorithms, reflecting the guarantees and pivot-selection strategies of each implementation. As expected BFPRT was largely unaffected because it uses the deterministic median-of-medians procedure, which guarantees a sufficiently balanced pivot regardless of the input arrangement.

Introselect experienced only a minor-to-moderate overall slowdown, with runtime changing by 3.7x and comparisons by 12x. This is consistent with its hybrid design. It initially behaves like randomized quickselect and is therefore vulnerable to poor partition sequences, but its depth limit eventually activates the BFPRT fallback. The fallback fired in 66.67% of runs on the adversarial input generated specifically against introselect, limiting the damage and preventing the algorithm from exhibiting the quadratic behavior seen in unprotected quickselect and std::nth_element.

Floyd-Rivest was hit hard by its own adversary. At $n = 10^5$ it needed 106 times its random-input comparisons (185 per element) and 16 times its running time, and the effect grew with size, from 17 times at $n = 10^3$. Its window is a contiguous block around position k rather than a random sample, so the adversary, which decides each comparison's outcome, can make the element selected inside the window a poor pivot every round. Unlike introselect, Floyd-Rivest has no depth limit or fallback, so nothing bounds this extra work. Its growth stayed well short of quadratic, though: quickselect on its own adversary reached about 18,750 comparisons per element. The attack is also specific to Floyd-Rivest: replayed against the other algorithms, the same array produced their usual comparison counts.

The largest effects were observed for randomized quickselect and std::nth_element. Quickselect required 894x the random-input runtime and 5,611x the random-input comparisons. std::nth_element required 908x the runtime and 6,265x the comparisons. These large multipliers are consistent with partition-based selection algorithms being driven toward highly unbalanced partitions by the McIlroy construction. Repeatedly processing partitions that remove only a small number of elements can produce near-quadratic comparison behavior. In particular, the results demonstrate that the practical worst-case behavior of std::nth_element depends on the standard-library implementation and its pivot

and fallback policies. Its average-case performance on ordinary inputs does not guarantee resistance to this adversarial construction.

5. What surprised us

5.1 Fewer comparisons doesn’t always mean less time (Michael)

One result that surprised me was the behaviour of BFPRT on the two-value input. I expected the duplicate-heavy input to not really change the runtime or comparison counts. Instead, BFPRT performed approximately 1.18 times as many comparisons as it did on the random input, but required only about 0.81 times as much time. At first this seemed contradictory. Even if I was wrong in my initial assumption, I would at least expect the runtime and comparisons to move in the same direction.

This result prompted me to research branch prediction and other possible explanations for the difference between the comparison count and the measured runtime. The number of comparisons is only one part of total execution time: the algorithm also spends time on partitioning, swapping, recursion, and moving data through memory. The two-value input may have produced more regular partitioning behaviour than the random input. Since the values are repeated so frequently, the outcomes of conditional tests may have followed more predictable patterns, such as repeatedly finding values equal to the pivot.

This could have made the control flow easier for the processor to predict. Better branch prediction would reduce the time lost when the processor guesses the wrong execution path. The repeated values may also have reduced the amount or complexity of data movement during partitioning, even though the comparison counter increased. Other possible causes include differences in cache behaviour, memory access patterns, recursion paths, and the cost of handling equal values. As a result, BFPRT may have completed more logical comparisons while carrying out the surrounding operations more efficiently.

This is only a possible explanation, because the benchmark counted comparisons and measured elapsed time, but did not directly measure branch mispredictions, cache behaviour, or memory traffic. Therefore, I cannot conclude that improved branch prediction was definitely responsible. However, the finding was useful because it showed me that comparison count is not a complete measure of algorithm performance. It prompted me to look beyond the algorithm’s theoretical operation count and consider how the implementation interacts with the hardware. This made the relationship between theoretical complexity and practical runtime seem more complicated than I initially assumed.

5.2 Floyd-Rivest does more work on reverse-sorted input than on random input (Ben)

What surprised me was Floyd-Rivest on reverse-sorted input. In P9 I expected both sorted and reverse-sorted input to help it, because a window of an ordered array holds closely ranked values. On sorted input this held exactly: 1.002 comparisons per element at every rank. On reverse-sorted input, however, it used 2.56 per element at the median against 1.57 on random input, and 2.00 against 1.00 at the minimum.

To find out why, I traced each round with a small program (benchmarking/trace_fr.cpp) at n = 10^7. On random input at the median, the range shrank from 10 million to 5.11 million and then to about 196,000. On reversed input it stayed at 5 million for three rounds. The cause is the window's safety margin, sd, which is measured in ranks within the sample. In a random sample each rank stands for about 430 elements, so the pivot misses the target by about 100,000, leaving the target near one end of what remains. In an ordered window the values are consecutive, so the margin shrinks to about 611 elements. Sorted order cancels it out; reversed order flips its direction, so the pivot lands just short of the target each round (611, 371, then 131 elements), giving four passes and about 2.5n comparisons. Sampling the top-level window at random confirmed this: reverse-sorted input dropped to 1.57n, matching random input, while sorted input rose to 1.53n. Floyd-Rivest's efficiency depends entirely on its window behaving like a random sample.

6. What we'd do next

The results opened several questions that would be useful to investigate further. First, we would examine whether a randomized-window version of Floyd–Rivest could preserve its low comparison count on random input while behaving more consistently on structured inputs such as organ-pipe, sorted, and reverse-sorted data. In particular, we would compare whether it can remain close to approximately 1.5n comparisons on random inputs without becoming vulnerable to particular input arrangements or sampling outcomes. This would help determine whether the algorithm's performance depends too heavily on its current deterministic sampling and narrowing strategy.

We would also use hardware performance counters to investigate cases where comparison counts do not explain the measured runtime. The two-value BFPRT result, where more comparisons took less time, raised questions about branch prediction, cache misses, memory traffic, and the number of swaps. Measuring branch mispredictions and cache behaviour could show whether the duplicate-heavy input produces more predictable control flow or more efficient memory access. This would replace speculation about microarchitectural effects with direct evidence.

Finally, we would add a std::nth_element baseline compiled with libstdc++, alongside the existing libc++ implementation. The current results use Apple's libc++ implementation, and its behaviour under the McIlroy adversary may differ substantially from libstdc++. Comparing both implementations would show whether the observed adversarial behaviour is a general property of std::nth_element or a consequence of a particular standard-library implementation. It would also make the results more useful when comparing against environments commonly used with GCC and Linux.

7. Individual contributions

7.1 Michael Lunn

Beyond implementing my primary selection algorithm, I contributed to the design and development of the benchmarking infrastructure. I helped integrate the algorithms into a common registry, added correctness checks, and investigated whether the benchmark was actually calling the group's implementations. I also worked on the plotting and data-analysis tools, including plots for runtime, comparisons, fallback activations, rank sensitivity, and string-comparison performance. I processed the benchmark results and investigated unexpected findings, such as the behaviour of std::nth_element on organ-pipe inputs, the effect of the McIlroy adversary, and the difference between comparison counts and runtime on duplicate-heavy inputs. I also contributed to the report by completing most of sections 1 and 4, and parts of section 3.

7.2 Ben Hood

I was the primary author of the Floyd-Rivest algorithm. I set up the report's structure and wrote section 1.4, the Floyd-Rivest predictions in 2.2, as well as sections 2.1, 3.4, and 3.5. I investigated why Floyd-Rivest did more work on reverse-sorted input than on random input, tracing its pivot error round by round and testing the explanation with a random-window run, and wrote this up as section 5.2. I also tested what happens without its window shift, which became part of the video's deep dive. I contributed to section 8, checked the repository for bugs and fixed the one found in Floyd-Rivest, and confirmed that the build and README instructions work. I helped build the slides and recorded and edited the video, apart from Michael's deep dive.

8. AI use

8.1 AI Use Disclosure

8.1.1 Michael Lunn

Claude was used in the initial stages of this project, coming up with the idea to specifically test selection algorithms. It created the initial implementations of the algorithms as well as the complete benchmarking suite and input generators.

Before running the experiments, I used Claude to generate a predictions worksheet, which I used to check that I was considering the relevant questions and forming reasonable expectations about the algorithms before testing them.

Throughout the project, I used GitHub Copilot to assist with implementing and modifying the benchmarking code, including plotting functionality and scripts for processing the results. I used Claude for research and to improve my understanding of algorithm behaviour, benchmarking results, and possible explanations for unexpected performance patterns.

Claude also helped me draft and refine sections of the report. I remained responsible for selecting the experiments, interpreting the results, checking the generated code and explanations, and writing the final conclusions.

8.1.2 Ben Hood

I used Claude throughout my work on Floyd-Rivest. Early on, I used it to explain the selection problem and the algorithm, including some worked examples. Claude then wrote the initial standalone implementation of the algorithm. The group later refactored it into the shared templated version. Claude also helped write the debug file used in section 5.2.

Later, Claude reviewed the repository read-only and suggested changes, which I applied myself. It helped draft the report template and sections 1.4, 3.4, 3.5, 5.2 and 8.3, and helped me interpret the results. I was responsible for deciding which suggestions to keep, checking drafts against the code and the CSVs, and the final wording. Claude was used to draft the presentation slides.

8.2 Where AI was wrong, unhelpful or misleading

One early problem was that the AI-generated benchmarking tool implemented its own versions of the algorithms rather than using the implementations written by the group. This meant that the benchmark could appear to work even if one of our actual algorithms was incorrect. We discovered the problem by intentionally breaking an algorithm and observing that the benchmark tests still passed. This showed that the benchmark was not exercising the code we intended to evaluate. We first ensured our algorithms used generic types so that they could support different inputs (e.g. strings) and then we corrected the infrastructure so that it called the group's registered implementations. We reviewed the benchmark and algorithm integration more carefully before continuing.

A second misleading suggestion involved the McIlroy adversarial inputs. The AI suggested testing BFPRT on a “killer” array generated for another algorithm and presenting this as evidence of where BFPRT excels. This misunderstood the purpose of the adversary. A killer input is designed to exploit the pivot and partition behaviour of a particular algorithm, so an adversary generated for quickselect cannot automatically be treated as a meaningful BFPRT test. We corrected this by generating adversarial inputs separately for each target algorithm and by clearly distinguishing between an algorithm’s own adversary and the replay of that adversary against other algorithms.

A third issue occurred with the fallback plots. The AI initially aggregated fallback counts using the median, which made the graph display zero even though some introselect runs had triggered the BFPRT fallback. This was misleading because fallback events were rare, so the median number of fallbacks was zero despite non-zero events being present in the raw data. We checked the CSV files directly and calculated the percentage of runs in which a fallback occurred. This showed that the fallback had fired in some adversarial runs. We therefore treated the graph as showing median fallbacks per run rather than interpreting a zero median as proof that no fallback had occurred. This reinforced the importance of checking raw benchmark data rather than relying only on AI-generated plots or summaries.

A fourth problem was an inaccuracy when using AI to debug. Claude reported that it “couldn’t break the window maths” in Floyd-Rivest after checking only that the window always contains k, not that it is always smaller than the range. A later test found a 2-element case where the window is the whole range, so the recursion never ends. This is unreachable with our experimental defaults, so no results are affected, and I added one-line to fix it. The lesson was to double check what a verification actually covered.

8.3 AI in group work

AI helped us understand each other’s algorithm well enough to review it and write about it. Ben used it to understand and explore the median of medians, and Michael for Floyd-Rivest. We also both used it to benchmark results and explain unexpected patterns, meaning we could implement an infrastructure of a size we could not have written ourselves. It lets us spend most of the time on experiment design and analysis, which is what holds the most weight.

However, the testing harness Claude first generated contained its own copies of the algorithm, meaning it appeared to work correctly without testing our implementation. This is more concerning in a group setting where each member could assure someone else had checked the shared infrastructure. AI-generated explanations also risked spreading incorrect information across the group. For example, Claude initially described Floyd-Rivest as using a two-pivot, three-way partition, which is not how it works. Since Section 1.4 was shared, this misunderstanding could have affected both our understanding and the final report.

Overall, AI let our two-person group build infrastructure of a size we could not have written ourselves in four weeks. The cost was that every generated component needed one of us to check it against the code or the raw data, rather than blindly trusting each other’s AI-assisted work.

References

Blum, M., Floyd, R. W., Pratt, V., Rivest, R. L., & Tarjan, R. E. (1973). Time bounds for selection. *Journal of Computer and System Sciences*, *7*(4), 448–461. https://doi.org/10.1016/s0022-0000(73)80033-9

Idea Behind. (2022, November 3). *What is median of medians algorithm for selection problem?* YouTube. https://www.youtube.com/watch?v=BA222oONWco

McIlroy, M. D. (1999) A killer adversary for quicksort. *Software.* 29(4) 341 - 344. https://dl.acm.org/doi/10.5555/311868.311871

Musser, D. R. (1997). Introspective sorting and selection algorithms. *Software: Practice and Experience*, *27*(8), 983–993. https://doi.org/10.1002/(sici)1097-024x(199708)27:8<983::aid-spe117>3.0.co;2-#